

4 [2 5 7 0]

$$A^k[i,j] = \min \{ A^{k-1}[i,j], A^{k-1}[i,k] + A^{k-1}[k,j] \}$$

→ code = for k=1 to n for i=1 to n for j=i to n

for (k=1; k<=n; k++)
for (i=1; i<=n; i++)
for (j=i; j<=n; j++)

This will be = 7+8 = [4 5] 0 A = [4 5] 0 A

generates Matrix
 $A[i,j] = \min(A[i,j], A[i,k] + A[k,j])$

$$B = 3 + 2 = [4, 1] \quad 0A + [4, 3] \quad 0A < 8 = [5, 3] \quad 0A$$

$\Theta(n^3) \leftarrow$ Time complexity

26 2 mod 26 (22)
[0 0 8 2]
[0 7 8 5]

→ Dijkstra's Algorithm may work or may not work in case of negative edges.

There is another soln for this single source shortest path problem that is Bellman Ford's Algorithm.

→ Pseudo code for Dijkstra's Algorithm

source vertex $\rightarrow$ set V (all vertices excluding source 's')
 $\text{dist}[s] \leftarrow 0$ (distance to source vertex is zero)

for all $v \in V - \{s\}$ do $\text{dist}[v] \leftarrow \infty$ (set all other distances to ∞)

other vertex $\leftarrow S \leftarrow \emptyset$ (S the set of visited vertices is initially Empty)

(individual vertex v) $\leftarrow Q \leftarrow V$ (Q the set of queue initially contains all vertices)

while $Q \neq \emptyset$ (While the queue is Not Empty)

do $u \leftarrow \text{mindistance}(Q, \text{dist})$ (select the element of Q with the min. distance)

$S \leftarrow S \cup \{u\}$

for all $v \in \text{neighbours}[u]$ (Add u to list of visited vertices)

do if $\text{dist}[v] \geq \text{dist}[u] + w(u, v)$

then $\text{dist}[v] \leftarrow \text{dist}[u] + w(u, v)$

(if desired, add traceback code)

return dist

(if New shortest path found)

OR Queue

$Q \rightarrow$ set Q to contain All vertices V.

(Q will act as our Working Set (or priority Queue) from which we will select the Next vertex with the smallest distance)

set u to the vertex in Q with the smallest distance in the 'dist' array

this operation $u \leftarrow \text{mindist.}(Q, \text{dist})$ finds the closest unvisited vertex to the source 's'.

(set New value of Shortest path)

starting

vertex (let it be 1)

Date:

apsa

→ void DFS(int u)

{
if (visited[u] == 0)

{
print("%d", u);
visited[u] = 1;

for (v = 1; v <= n; v++)
if (A[u][v] == 1 && visited[v] == 0)

{
DFS(v);
}

}

}

}

minimum cost spanning tree

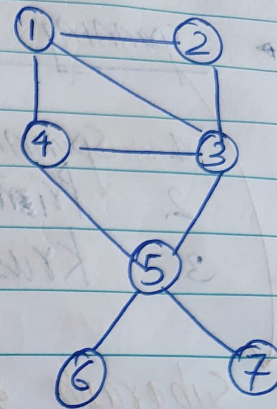
4, 1, 3, 5, 6, 7, 2

Program for DFS :-

U ↓

	0	1	2	3	4	5	6	7
0								
1		0	1	1	1	0	0	0
2		1	0	1	0	0	0	0
3		1	1	0	1	1	0	0
4		1	0	1	0	1	0	0
5		0	0	1	1	0	1	1
6		0	0	0	0	1	0	0
7		0	0	0	0	1	0	0

← V



No. of vertices = 7

8x8 matrix

Adjacency matrix for representing Graph

	1	1	1	1	1	1	1	
Visited	0	0	0	0	0	0	0	
	0	1	2	3	4	5	6	7

(I am Not using 0th row & 0th column)
So 0 is placed for all indices at index 0

enqueue (u, v);

// for loop ends

// while loop ends

Time complexity = $O(n^2)$ [on using Matrix] $\downarrow$ No. of edges

Time depends on $|V| + |E| \Rightarrow O(|V| + |E|)$

No. of vertices

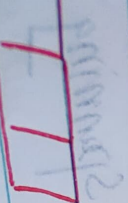
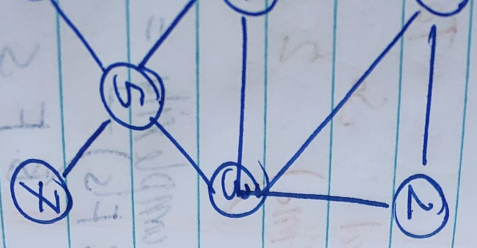
→ Depth First Search (graph traversal method) [uses stack data structure]

This is similar to preorder traversal of Binary trees.

1] visited (visiting a vertex)

2] Exploring (visiting all the adjacent vertices of a particular vertex)

let us understand



10	01	1
5	6	7

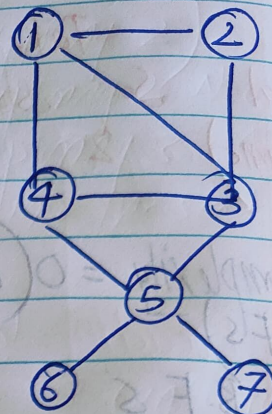
2) 5, 7, 3, 6, 4, 2, 1

3) 2, 3, 1, 5, 4, 6, 7

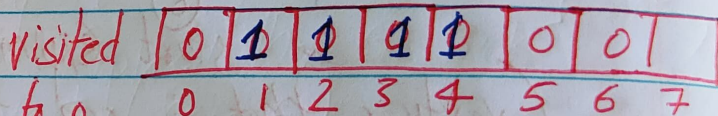
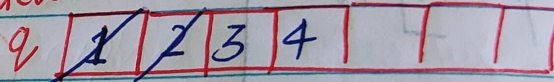
And so on.

Program for BFS

	0	1	2	3	4	5	6	7
0								
1		0	1	1	1	0	0	0
2		1	0	1	0	0	0	0
3		1	1	0	1	1	0	0
4		1	0	1	0	1	0	0
5		0	0	1	1	0	1	1
6		0	0	0	0	1	0	0
7		0	0	0	0	1	0	0



Queue



Starting vertex (Assume $i=1$)

void BFS (int i)

int u;

printf ("%d" i);

visited [i] = 1;

enqueue (q, i);

While (!is Empty (q))

{ u = dequeue (q);

for (v = 1; v <= n; v++)

{ if (A[u][v] == 1 &

visited [v] == 0)

printf ("%d" v);

visited [v] = 1;

calling function enqueue directly.

Ford Fulkerson Method

:

FORD-FULKERSON(G, s, t)

1 **for** each edge $(u, v) \in G.E$

2 $(u, v).f = 0$

3 **while** there exists a path p from s to t in the residual network G_f

4 $c_f(p) = \min \{c_f(u, v) : (u, v) \text{ is in } p\}$

5 **for** each edge (u, v) in p

6 **if** $(u, v) \in E$

7 $(u, v).f = (u, v).f + c_f(p)$

8 **else** $(v, u).f = (v, u).f - c_f(p)$

How to find MST using Kruskal's algorithm?

Below are the steps for finding MST using Kruskal's algorithm:

- Sort all the edges in non-decreasing order of their weight.
- Pick the smallest edge. Check if it forms a cycle with the spanning tree formed so far. If the cycle is not formed, include this edge. Else, discard it.
- Repeat step#2 until there are $(V-1)$ edges in the spanning tree.

How does Prim's Algorithm Work?

The working of Prim's algorithm can be described by using the following steps:

Step 1: Determine an arbitrary vertex as the starting vertex of the MST.

Step 2: Follow steps 3 to 5 till there are vertices that are not included in the MST (known as fringe vertex).

Step 3: Find edges connecting any tree vertex with the fringe vertices.

Step 4: Find the minimum among these edges.

Step 5: Add the chosen edge to the MST if it does not form any cycle.

Step 6: Return the MST and exit

MST-PRIM(G, w, r)

```
1  for each  $u \in G.V$ 
2       $u.key = \infty$ 
3       $u.\pi = \text{NIL}$ 
4   $r.key = 0$ 
5   $Q = G.V$ 
6  while  $Q \neq \emptyset$ 
7       $u = \text{EXTRACT-MIN}(Q)$ 
8      for each  $v \in G.Adj[u]$ 
9          if  $v \in Q$  and  $w(u, v) < v.key$ 
10              $v.\pi = u$ 
11              $v.key = w(u, v)$ 
```